

Neuro-Symbolic Planning and Scene-Graph-Based Reasoning for No-Code Robotic Task Programming

Samuele Sandrini^{*,a,1}

^a*Institute of Intelligent Industrial Technologies and Systems for Advanced Manufacturing, National Research Council of Italy*

Research proposal draft – July 21, 2026

Abstract

Industrial robotic programming still requires expertise to translate process requirements into executable behaviors under changing products, tools, constraints, and human-robot interaction conditions. Foundation models support natural-language specification and semantic reasoning, but remain insufficient as standalone planners where correctness, traceability, timing, and safety are critical. Conversely, symbolic planning provides explicit models, efficient planners, and validation mechanisms, but its adoption is limited by the effort required to align planning domains with the robotic software stack and the perceived scene.

This proposal investigates a neuro-symbolic framework for no-code robotic task programming in industrial environments. Large language models are used to interpret operator requests, extract task-relevant information, and support the generation or repair of formal planning models, while symbolic planners and validation tools remain responsible for plan synthesis, consistency checking, and executability assessment.

The research is organized into three goals that define a progressive research path. First, given an available planning domain, natural-language requests are transformed into validated planning problems, solved by a planner portfolio, and grounded into executable behaviors by discovering and analyzing the ROS 2 computational graph. Second, the assumption of an available planning domain is relaxed by deriving skill-grounded models from multimodal sources, including software interfaces, process documentation, code, images, and knowledge bases. Third, scene graphs for mechanical assemblies are used as relational representations to infer disassembly constraints, enrich planning contexts, monitor execution, and support replanning under uncertainty.

The expected outcome is a neuro-symbolic architecture that reduces manual modeling effort while preserving robustness, transparency, and execution reliability in industrial robotic applications.

Keywords: *Neuro-symbolic planning, large language models, robotic task planning, PDDL, scene graphs, skill registry, behavior trees, PlanSys2, disassembly planning, task and motion planning, human-robot collaboration*

1. INTRODUCTION

LARGE language models (LLMs) and foundation models have been increasingly adopted in robotics to map natural-language goals into executable behavior, including task decomposition, code generation, and context-aware action selection. Early embodied-control approaches showed that code-trained LLMs can synthesize robot-centric programs that call perception and control APIs, implement feedback loops, and parameterize waypoint-based or reactive policies from natural-language commands [1]. Other approaches shifted from direct code generation to structured task planning: ProgPrompt generates situated robot task plans through programmatic prompting [2], SayCan grounds language-level action proposals in the affordances of pretrained robot skills [3], and Text2Motion combines language-based planning with feasibility-aware task and motion reasoning [4]. Subsequent embodied multimodal models, such as PaLM-E and RT-2, further reduced the separation between perception, language, and action by conditioning robot behavior on visual, linguistic, and embodiment-specific inputs [5, 6]. This line of work is now extending toward Vision-Language-Action (VLA) models, which couple visual and linguistic inputs with action prediction for generalist robot control [7, 8], and World Action Models (WAMs), which use predictive representations of future observations to score, guide, or generate actions [9].

Despite these advances, industrial robotic applications impose requirements that are difficult to satisfy by relying on LLMs/VLMs/VLAs as standalone planners and executors. Manufacturing cells require repeatability, traceability, verifiability, safety-aware execution, and predictable performance under temporal, spatial, and human-robot interaction constraints [10–15]. Although LLMs are effective at leveraging commonsense and semantic knowledge for natural-language interaction and task-level interpretation, several planning benchmarks show that they still struggle with formal action reasoning, long-horizon dependencies, spatio-temporal constraints, and plan

validation [16–19].

These observations motivate the integration of LLMs not as substitutes for formal planners, but as providers of semantic task knowledge, candidate abstractions, and language-based interfaces within model-based, verifiable planning loops [20, 21].

Conversely, classical automated planning provides a complementary set of strengths. Planning formalisms such as PDDL and PDDL2.1 offer explicit semantics for actions, preconditions, effects, temporal constraints, numeric fluents, and plan metrics [22, 23]. Other modeling languages and frameworks, including ANML and RDDDL, extend this perspective toward richer temporal structures and stochastic decision processes [24, 25]. Depending on the problem class and the planner, classical approaches can provide soundness, completeness, or optimality guarantees, and can optimize criteria such as makespan, energy, cost, risk, or task-allocation efficiency. Temporal planners such as OPTIC handle preferences and time-dependent costs [26], while recent Optimization Modulo Theories approaches address optimal temporal planning through abstract action scheduling [27]. In collaborative manufacturing, planning and optimization have also been used to account for human-aware task allocation and scheduling with safety-related interaction effects [15]. Moreover, timeline-based and stochastic planning approaches explicitly model flexible temporal structures, uncontrollable durations, and uncertainty in action outcomes [28–30].

However, these classical approaches typically require a substantial symbolic or mathematical modeling effort. An AI-planning expert must encode the problem under consideration into an effective formal model, e.g., using one of the above-mentioned formalisms such as

^{*}Corresponding author: samuele.sandrini@stiima.cnr.it.

Proposal prepared for a research position in the context of Plan4ARI – Planning for Artificial Robotic Intelligence and Beyond.

PDDL, ANML, or RDDDL [22–25]. This process often requires close interaction between planning experts and process experts, and is rarely carried out by the same engineers who configure the robotic cell, develop ROS 2 nodes, integrate perception modules, or define the industrial process. As a consequence, classical planning remains difficult to deploy in high-mix, low-volume production, where process variants, tool availability, robot skills, fixtures, objects, and operator requests change frequently. Thus, the gap is not only between natural language and a planning problem, but also between the running robotic software stack and the symbolic plan generated from the formal problem model.

These observations motivate the integration of LLMs and formal planners in order to exploit the strengths of both: LLMs provide natural-language interaction with users, semantic interpretation of task requests, and support for task-level abstraction, while model-based planners provide explicit reasoning, verification, and optimization capabilities [20, 31]. Recent neuro-symbolic approaches try to bridge this gap by combining LLMs with formal planning tools. LLM+P translates natural-language planning tasks into PDDL problems and delegates plan synthesis to classical planners [32]. Other works use LLMs to construct or refine symbolic world models [33], generate planning domains from textual descriptions [34], or incrementally produce PDDL models from minimal natural-language input, as in NL2Plan [35]. Surveys on LLM-based planning formalization highlight the promise of this direction while also emphasizing unresolved issues in model correctness, validation, grounding, and user interaction [31]. In robotics, Safe Planner uses an LLM to map natural-language instructions into PDDL goals and combines them with closed-loop visual observations and safety-aware skill selection [36]. However, many existing LLM-planner systems still rely on a pre-defined or externally curated planning domain, or address domain generation only under simplified textual assumptions, or from precise textual description, leaving open the problem of deriving executable planning models from the actual robotic software stack, available skills, process documentation, and online perceptual state.

Another key open challenge is state grounding. Robotic planning cannot be decoupled from scene understanding: the initial state, action preconditions, and action effects must be estimated from perception and updated during execution. Scene graphs offer a structured representation of objects, attributes, and spatial or functional relations, and are increasingly used as an intermediate layer between perception and planning. SayPlan grounds LLM-based planning in large-scale 3D scene graphs [37], while domain-conditioned scene graphs explicitly map scene understanding to planning predicates and improve state-grounded task planning [38]. Similarly, VLM-TAMP uses vision-language models to generate subgoals that guide long-horizon task and motion planning while preserving geometric feasibility through a symbolic-geometric planner [39]. Moreover, existing approaches often focus on semantic descriptions of open environments, while their use for reasoning about relations among mechanical parts in industrial assemblies, e.g., to infer disassembly constraints, remains limited.

This research proposal aims to address the research gaps identified above by developing a neuro-symbolic architecture for no-code robotic task planning and execution in industrial settings. The first research direction targets the gap between symbolic plans and executable robot behaviors. It investigates an LLM-assisted symbolic planning framework that starts from an available PDDL domain and uses the ROS 2 computational graph to relate abstract planning actions to the skills, interfaces, and execution resources exposed by the robotic system. Within this framework, operator requests are transformed into validated planning problems, solved through a suitable planner selected from a portfolio, and checked before execution through a ROS 2-oriented control layer.

The second research goal relaxes the assumption that a planning domain is already available. While existing work such as [33] con-

structs planning domains with LLMs from detailed natural-language descriptions of actions, the focus of this research goal is different. It focuses on extracting a skill registry from multimodal sources of information, including the running computational graph, available software libraries, process documentation, and knowledge bases. This registry is then used to support the generation of action schemas, parameters, preconditions, and effects grounded in the actual robotic setup, reducing the manual knowledge-engineering effort required to deploy automated planning in changing industrial cells. Moreover, the construction of such a skill registry makes the approach less dependent on a specific modeling language such as PDDL, enabling the selection of the most appropriate formalism depending on the requirements of the task.

The third research direction focuses on scene understanding through scene graphs as both offline context representations and on-line execution monitors. In particular, it studies how scene graphs can support the understanding of relations among mechanical parts in industrial assemblies, enabling LLM-based inference of disassembly constraints or, more generally, precedence constraints that enrich the input context of the LLM-assisted planning framework. During execution, scene graphs support state grounding, precondition checking, effect validation, and robust replanning, while enabling scene-conditioned contingent planning under perceptual uncertainty.

The proposal is organized as follows. Section 2 describes the methodological foundations of the proposed research directions, together with the preliminary concepts required to frame them. Section 3 discusses the implementation of the proposed framework and the evaluation protocol planned for its assessment. Section 4 summarizes the expected scientific contributions of the research proposal, while Section 5 highlights the alignment of the proposed research with the objectives and thematic priorities of the call. Finally, Section 6 concludes the proposal by summarizing its expected impact and future research perspective.

2. METHODOLOGY

This section describes the methodological framework underlying the research directions addressed in this research proposal.

2.1. Research Goal 1: From Natural-Language Goals to Executable Symbolic Plans

Classical and temporal planning preliminaries. A classical planning task is a pair $P = (D, I)$, where D is a planning domain and I is a planning problem, typically encoded in PDDL as a domain file and a problem file, respectively. The domain D defines a set of predicate symbols, numeric fluents, and action schemas. Each action schema is specified by typed parameters, preconditions, and effects expressed over predicates or fluents. The problem instance is defined as $I = (\mathcal{O}, s_0, s_G)$, where $\mathcal{O}$ is a finite set of objects, s_0 is the initial state, and s_G is the goal condition. Grounding the action schemas over $\mathcal{O}$ induces a transition system in which states are sets of ground atoms and fluent assignments, applicable actions are those whose preconditions hold in the current state, and the transition function updates the state according to the action effects. A plan is an applicable sequence of grounded actions that reaches a state satisfying s_G . In the temporal setting, actions may also have durations. A grounded timed action a_i is associated with a start time t_i and a duration d_{a_i} , yielding a plan

$$\pi = \{(t_i, a_i, d_{a_i})\}_{i=1}^N. \quad (1)$$

The plan is valid if all preconditions and effects are respected along the induced execution trace and the final state s_N satisfies the goal condition, i.e., $s_G \subseteq s_N$. When optimizing for execution time, the objective is to minimize the makespan:

$$\pi^* = \arg \min_{\pi} \left(\max_{i \in [1, N]} (t_i + d_{a_i}) \right). \quad (2)$$

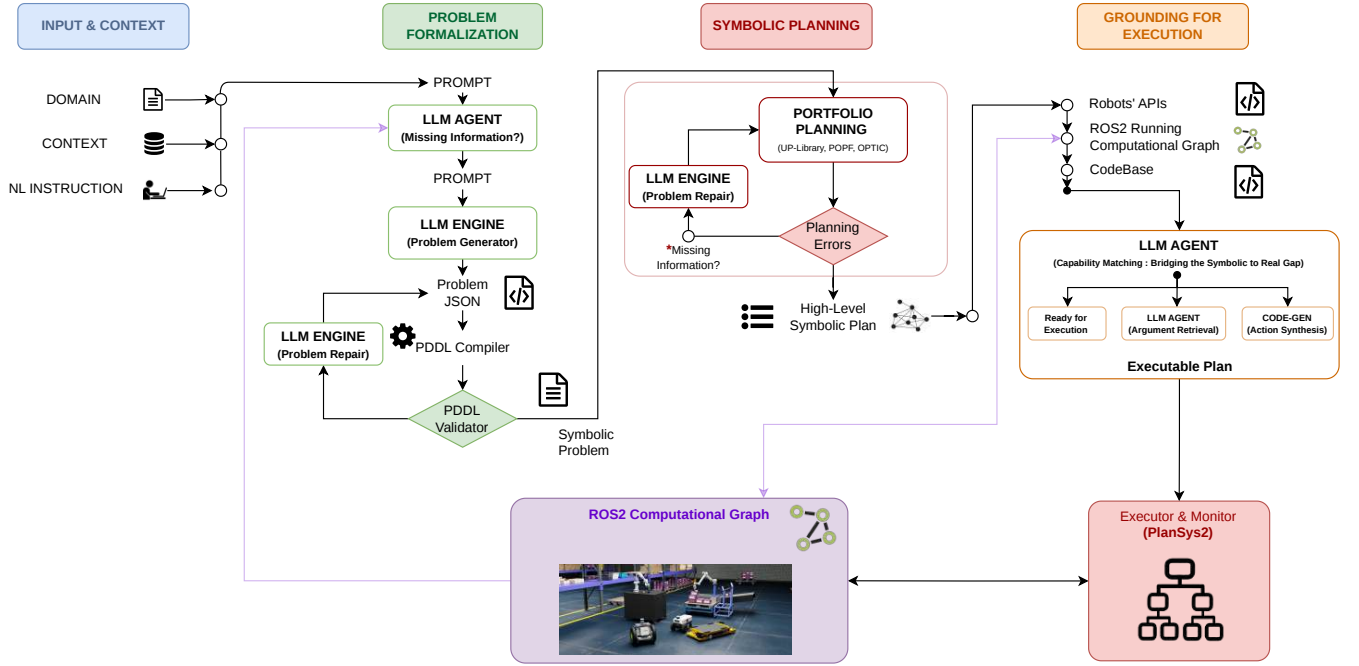


Figure 1. Overview of the proposed neuro-symbolic framework for transforming natural-language operator requests into executable robotic plans.

We focus on how neuro-symbolic reasoning can support the transition from operator-level task specifications to formal planning models and executable robotic behaviors.

Proposed problem formulation:

Assumptions. We assume that a planning domain D is available and encoded in PDDL, and that the framework has access to the running ROS 2 system $\mathcal{R}$. During the workflow, LLM-based nodes equipped with ROS 2 introspection tools can query the runtime computational graph $\mathcal{G}_{ROS}$ on demand, for example to discover available nodes, topics, services, actions, parameters, interfaces, or to verify the availability of required PlanSys2 action executors.

Problem statement. Given the domain D , an operator natural-language request $\mathcal{L}$, an optional textual context $\mathcal{C}$, and access to $\mathcal{R}$, the objective is to synthesize and execute a symbolic plan that is both valid with respect to the planning model and grounded in the robotic system. Formally, the proposed framework implements the transformation

$$\mathcal{F} : (D, \mathcal{L}, \mathcal{C}, \mathcal{R}) \rightarrow (I, \pi, \Gamma) \quad (3)$$

where $I = (\mathcal{O}, s_0, s_G)$ is a valid planning problem, π is a symbolic plan for (D, I) , and Γ is an execution-grounding map that associates symbolic actions in π with executable PlanSys2/ROS 2 behaviors.

Proposed approach.

The workflow reported in Fig. 1 is modeled as an agentic workflow graph

$$\mathcal{A} = (\mathcal{N}, \mathcal{E}, \mathbf{x}), \quad (4)$$

where $\mathcal{N}$ is the set of computational nodes, $\mathcal{E}$ defines the control flow among them, and $\mathbf{x}$ is a shared structured state. The state stores the domain schema, the operator request, optional context, ROS 2 evidence, generated problem specifications, validation reports, planner outputs, selected plan, and execution-grounding diagnostics. Each node implements a typed transformation of the shared state. Some nodes are deterministic, such as PDDL compilation and validation, while others are LLM-based and tool-augmented, but constrained by the domain schema and by external validation procedures.

Problem Formalization. The first stage extracts from D a domain signature

$$\Sigma_D = (\mathcal{T}, \mathcal{P}, \mathcal{F}, \mathcal{A}_D), \quad (5)$$

where $\mathcal{T}$ is the type hierarchy, $\mathcal{P}$ the set of predicate symbols, $\mathcal{F}$ the set of numeric fluents, and $\mathcal{A}_D$ the set of action schemas. A grounding node then checks whether the information contained in $\mathcal{L}$ and $\mathcal{C}$ is sufficient to instantiate a planning problem compatible with Σ_D . When information is missing or ambiguous, the node queries $\mathcal{G}_{ROS}$ through ROS 2 introspection tools, retrieving available nodes, topics, services, actions, parameters, interfaces, and known execution resources. Human confirmation is requested only when the required information cannot be reliably inferred from the robotic system or when multiple inconsistent interpretations remain possible.

Once the request is sufficiently grounded, a problem-generation node produces an intermediate structured representation, denoted as σ , in JSON format. This representation includes the problem objects, initial facts, numeric initial values, goal condition, assumptions, unknowns, and possible domain gaps. A deterministic compiler then converts σ into a PDDL problem I . The use of an intermediate representation decouples LLM-based semantic interpretation from PDDL syntax generation, allowing the generated problem to be checked before being passed to a planner.

The compiled problem is validated by two complementary validators: VAL [40] and the Unified Planning framework [41]. The validation step checks syntactic correctness, predicate and fluent consistency, object typing, action applicability, and compatibility with the domain D . If validation fails, the corresponding diagnostics are inserted into the shared state and passed to a repair node, which revises σ under the constraint that all generated symbols must belong to Σ_D . The generate-compile-validate-repair loop is repeated until a valid problem is obtained or a failure condition is reached.

Symbolic Planning. Given a valid planning task $P = (D, I)$, the framework invokes a portfolio of symbolic planners. Each planner either returns a candidate plan π_k or a planning diagnostic. If planning fails, the error reports are passed to the repair node to revise the generated problem while preserving the operator intent. If one or more candidate plans are found, a plan-selection node chooses the most promising plan

$$\pi^* = \arg \min_{\pi_k \in \Pi} J(\pi_k), \quad (6)$$

where J denotes a plan cost function, encoding criteria such as plan length, makespan, execution cost, number of required resources, or an LLM-based ranking criterion.

Grounding for Execution. Symbolic validity alone does not guarantee robotic executability. Therefore, the final stage checks whether every grounded symbolic action $a_i \in \pi^*$ can be executed by the current PlanSys2/ROS 2 system. Let $\mathcal{B}$ denote the set of executable behaviors exposed by running PlanSys2 action executors and ROS 2 interfaces. The framework searches for a grounding map

$$\Gamma : a_i \rightarrow b_i, \quad b_i \in \mathcal{B}, \quad (7)$$

for each action a_i in the selected plan. This check verifies both the availability of a compatible PlanSys2 action and the correspondence between the symbolic action arguments and entities, parameters, or resources observable in $\mathcal{G}_{ROS}$.

If an action executor is available but some symbolic arguments are not grounded, the framework attempts to recover the missing bindings by querying the computational graph and the available robotic interfaces. If no executor is available, the framework checks whether the current ROS 2 graph exposes sufficient lower-level capabilities to implement the missing action. When this is possible, a specialized code-generation agent is invoked to synthesize a PlanSys2-compatible action executor, using the symbolic action schema, its parameters, and the relevant ROS 2 interfaces as input. The generated executor is then integrated into the running system and the grounding check is repeated. If the action cannot be implemented from the available interfaces, the missing capability is reported to the operator.

After all actions in π^* are grounded, the domain, problem, and selected plan are loaded into PlanSys2. The symbolic plan is then converted into a Behavior Tree and executed through the PlanSys2 execution layer. The resulting architecture therefore closes the loop from natural-language task specification to symbolic planning and robot execution, while preserving explicit validation, repair, and executability checks at each stage.

2.2. Research Goal 2: Robotic Skill Discovery and Grounding from Multimodal Data for Planning-Domain Generation

Research Goal 2 extends the previous direction by relaxing the assumption that a planning domain is already available. Instead of starting from a manually defined PDDL domain, the objective is to infer a planning-domain model from the capabilities exposed by the robotic system and by the information sources available around it. This direction addresses the knowledge-engineering bottleneck that arises when automated planning has to be deployed in changing industrial cells, where robot skills, interfaces, tools, and task constraints may evolve over time.

Assumptions. We assume that the framework has access to the running ROS 2 system $\mathcal{R}$ and to a set of heterogeneous multimodal information sources $\mathcal{M}$, including software documentation, process descriptions, robot manuals, code repositories, images, and structured knowledge bases. Unlike Research Goal 1, no complete planning domain D is assumed to be available.

Problem statement. Given $\mathcal{R}$ and $\mathcal{M}$, the objective is to construct a skill-grounded planning domain that can be used by the planning pipeline described in Research Goal 1. Formally, the proposed framework implements the transformation

$$\mathcal{F}_2 : (\mathcal{R}, \mathcal{M}) \rightarrow (\mathcal{S}, \hat{D}, \Phi), \quad (8)$$

where $\mathcal{S}$ is a robotic skill registry, $\hat{D}$ is a generated planning domain, and Φ is a grounding map that associates the action schemas of $\hat{D}$

with one or more robotic skills, ROS 2 interfaces, and execution resources available in the system. The generated domain $\hat{D}$ can then be used as the input planning domain for the neuro-symbolic planning framework introduced in Research Goal 1.

Proposed approach.

The workflow shown in Fig. 2 is organized around the construction of a skill registry. The first stage collects evidence about available robot capabilities from multimodal sources. This evidence is used to identify candidate skills, their input parameters, required resources, execution interfaces, and observable outcomes.

Each skill $s_j \in \mathcal{S}$ can be represented as a structured entry

$$s_j = (n_j, \mathcal{X}_j, \rho_j, \eta_j, \epsilon_j, \kappa_j), \quad (9)$$

where n_j is the skill name, $\mathcal{X}_j$ is the set of typed parameters, ρ_j denotes required resources or execution conditions, η_j denotes the associated ROS 2 or PlanSys2 execution interface, ϵ_j describes the expected symbolic effects or observable state changes, and κ_j encodes process-related constraints associated with the skill, objects, or object relations, such as precedence, safety, geometric, resource, or assembly/disassembly constraints.

Process constraints are extracted from the available multimodal sources and linked to the corresponding skills, objects, or relations. For example, a disassembly instruction such as “part A can be removed only after its four screws have been removed” can be represented as a precedence constraint and translated into planning preconditions for the corresponding removal action. In this way, the generated domain not only describes executable robot capabilities, but also captures the task-level constraints that determine their valid ordering.

Starting from the registry $\mathcal{S}$, the framework generates candidate planning-domain elements, including types, predicates, numeric fluents, and action schemas. For each candidate action schema $\hat{a}_j$, the system estimates its parameters, preconditions, and effects from the corresponding skill entry and from the contextual information available in $\mathcal{M}$. The resulting domain can be written as

$$\hat{D} = (\hat{\mathcal{T}}, \hat{\mathcal{P}}, \hat{\mathcal{F}}, \hat{\mathcal{A}}), \quad (10)$$

where $\hat{\mathcal{T}}$ is the inferred type hierarchy, $\hat{\mathcal{P}}$ the set of predicates, $\hat{\mathcal{F}}$ the set of numeric fluents, and $\hat{\mathcal{A}}$ the set of generated action schemas.

Since LLM-generated planning domains may contain inconsistent symbols, underspecified preconditions, or effects that are not supported by the robotic system, the generated domain is checked through validation and grounding steps. As in Research Goal 1, the proposed approach does not rely on an unconstrained end-to-end LLM prompt with generic tool access. Instead, domain generation is structured as an agentic state graph with explicit intermediate representations, validation steps, and repair loops. In particular, generated planning domains are checked through syntactic and semantic consistency tests and, when associated planning problems or test plans are available, through validation tools such as VAL and the Unified Planning framework. Different information-access strategies, including tool-augmented prompting, retrieval-augmented generation, multimodal retrieval, structured schema-based extraction, knowledge-graph grounding, and validator-in-the-loop repair, will be compared to assess how effectively the framework retrieves, interprets, and uses multimodal evidence relevant to the operator request.

The expected outcome of this research goal is therefore a skill-grounded domain-generation process that reduces the manual effort required to encode planning domains for robotic applications. By deriving symbolic actions from the skills, constraints, and interfaces actually available in the robotic cell, the approach aims to improve the portability of task-planning systems across different robots, tools, and industrial configurations.

Finally, as a possible future extension, we will investigate the integration of recent Task and Motion Planning and Scheduling approaches into the proposed framework, such as ScheduleStream [42], which

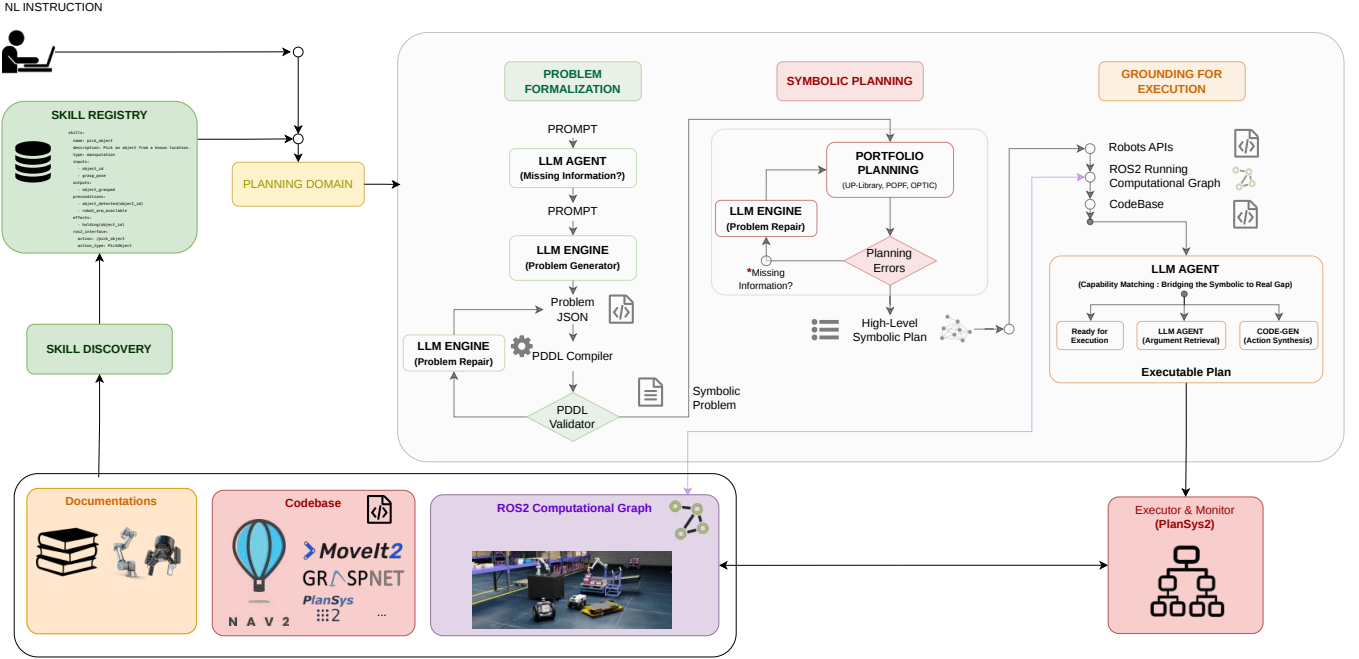


Figure 2. Overview of the proposed neuro-symbolic framework for robotic skill discovery and grounding toward planning-domain generation.

combines temporal planning with sampling-based operations and GPU-accelerated samplers for multi-arm task and motion planning.

2.3. Research Goal 3: From Scene Graph to Behavior Tree Execution for Disassembly

Behavior Tree preliminaries. A Behavior Tree (BT) [43] is formally defined as a directed rooted tree composed of internal *control-flow* nodes and leaf *execution* nodes. The root node has no parent, whereas all other nodes are connected through parent-child relations. BT execution is driven by a periodic *tick* signal, which is propagated from the root according to the semantics of the control-flow nodes until it reaches leaf nodes, thereby triggering their evaluation or execution. Each ticked node returns a status to its parent, typically one of SUCCESS, FAILURE, or RUNNING.

Common control-flow nodes include:

- *Sequence*: ticks its children from left to right and returns SUCCESS if all children succeed, FAILURE as soon as one child fails, and RUNNING while a child is still executing.
- *Fallback*: ticks its children from left to right and returns SUCCESS as soon as one child succeeds, FAILURE only if all children fail, and RUNNING otherwise.
- *Decorator*: typically has a single child and implements a policy δ that modifies the tick propagation or the returned status, for example by negating the child status, limiting the number of executions, or retrying a child until success.

Execution nodes are usually divided into:

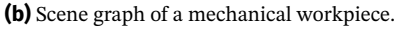
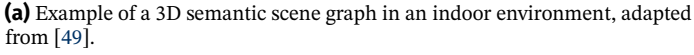
- *Condition*: checks whether a predicate holds and returns either SUCCESS or FAILURE, without returning RUNNING.
- *Action*: executes a behavior and returns SUCCESS if the behavior completes successfully, FAILURE if it fails, and RUNNING while execution is ongoing.

BTs have been widely adopted in robotics after their initial success in the video-game industry. Compared with finite-state machines and planning-domain languages such as PDDL, they provide a modular, reactive, interpretable, and maintainable representation of robot behavior [44]. However, BTs do not perform planning by themselves: the desired execution policy is explicitly encoded by the developer

in the tree structure. Their adoption has been further supported by mature software libraries such as BehaviorTree.CPP and py_trees, and by the fact that BT structures can be compactly represented in XML.

Scene Graph preliminaries. A Scene Graph (SG) can be formally represented as a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of nodes and $\mathcal{E}$ is the set of directed edges. Each node $v_i \in \mathcal{V}$ represents an entity in the scene, such as an object, part, region, or support surface, while each edge $e_{ij} \in \mathcal{E}$ encodes a relation from v_i to v_j . Equivalently, an SG can be expressed as a set of relational triplets $\langle v_i, r_{ij}, v_j \rangle$, where r_{ij} denotes the semantic, spatial, functional, or hierarchical relation between the two entities. Node attributes may further encode object categories, poses, geometric properties, states, affordances, or task-relevant features. In natural-language terms, an SG provides a compact symbolic description of what is present in the scene and how the scene entities are related. Scene Graph Generation (SGG) refers to the automatic construction of this representation from perceptual input, usually by detecting objects and predicting their pairwise relations [45]. Recent SGG methods have focused on improving the trade-off between accuracy and inference speed, as in REACT and REACT++ [46, 47], while robotics-oriented approaches such as MomaGraph introduce state-aware, task-driven scene graphs and train a vision-language model with reinforcement learning to predict task-oriented scene representations for embodied planning [48]. However, these approaches mainly target scene understanding in open household or embodied-agent environments. In this work, we investigate SGs for industrial mechanical assemblies as shown in Figure 3b, where the objective is to infer the semantic, spatial, and functional relations among assembled parts and use them to derive disassembly constraints, precedence relations, and execution-monitoring conditions for task-level planning. A preliminary SG representation for this setting may include nodes for mechanical parts and their affordances, fasteners, tools, contact surfaces, graspable regions, and access directions. Edges may encode relations such as connected-to, blocks, supports, requires-tool, must-be-removed-before, or accessible-from. This representation provides an explicit relational context for the LLM and can be transformed into symbolic facts or constraints used by the planner, for example to generate preconditions, validate action effects, or monitor execution progress.

In recent robotic task-planning approaches, scene understanding is



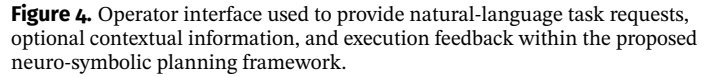
often delegated to the visual embeddings of general-purpose VLMs or to models fine-tuned for behavior generation. For instance, BTGenBot and BTGenBot-2 investigate the generation of executable Behavior Trees from task descriptions and available action primitives [50, 51], while recent multimodal extensions fine-tune compact VLMs to generate Behavior Trees from visual observations and natural-language instructions [52]. Although these approaches show the potential of language and vision-language models for robotic task generation, they still rely on learned latent visual representations that may be difficult to inspect, constrain, or adapt to industrial mechanical assemblies with domain-specific parts and relations.

The contribution investigated in this research goal is to introduce Scene Graphs as an explicit intermediate representation between perception and task-level reasoning. Instead of directly prompting a VLM with raw visual observations, the perceived scene is first abstracted into a structured graph

$$\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t, \mathcal{A}_t), \quad (11)$$

where $\mathcal{V}_t$ denotes the detected parts or objects, $\mathcal{E}_t$ their semantic, spatial, functional, or assembly relations, and $\mathcal{A}_t$ the associated node and edge attributes. This graph is then used as a compact symbolic context for LLM-based reasoning. The hypothesis is that replacing unstructured visual context with an explicit relational representation can reduce hallucinations, improve interpretability, and limit the need for fine-tuning general-purpose VLMs on highly specific mechanical components.

Two complementary research contributions will be investigated. The first one concerns Scene-Graph-conditioned Behavior Tree generation for disassembly tasks. Given a scene graph $\mathcal{G}_r$, an operator instruction $\mathcal{L}$, and a set of available robot skills or BT action nodes $\mathcal{B}$, the objective is to synthesize an executable Behavior Tree



$$\mathcal{F}_{BT} : (\mathcal{G}_t, \mathcal{L}, \mathcal{B}) \rightarrow \mathcal{T}_{BT}, \quad (12)$$

where $\mathcal{T}_{BT}$ is a Behavior Tree encoding the required disassembly strategy. This direction follows the spirit of BTGenBot-style behavior generation, but replaces direct visual prompting or VLM fine-tuning with a two-stage pipeline: scene perception to Scene Graph, and Scene Graph to Behavior Tree through an LLM. An alternative variant will also be explored, in which an LLM or multimodal model is fine-tuned to directly consume Scene Graph representations as structured input for BT generation.

The second contribution concerns the integration of Scene Graphs into the neuro-symbolic planning framework developed in Research Goals 1 and 2. In this case, the Scene Graph is not only used to generate a Behavior Tree, but also to enrich the planning context with explicit object relations. Formally, the planning context can be augmented as

$$\mathcal{C}' = \mathcal{C} \cup \psi(\mathcal{G}_t), \quad (13)$$

where ψ denotes a graph-to-symbolic-context mapping that extracts task-relevant symbolic facts from the Scene Graph $\mathcal{G}_t$, such as object-part relations, support relations, fastening relations, occlusions, accessibility constraints, or disassembly precedences. These facts can be used to refine planning problems, generate or validate preconditions, monitor execution effects, and trigger replanning when the observed scene differs from the expected symbolic state.

The evaluation will compare Scene-Graph-conditioned reasoning against approaches that rely more directly on VLM-based scene understanding. The comparison will focus on the correctness of inferred disassembly constraints, Behavior Tree validity, planning success, execution-monitoring accuracy, and robustness to visually similar or domain-specific mechanical parts.

Finally, as a possible future extension, we will investigate probabilistic and dynamic Scene Graphs for online scene monitoring, including representations that explicitly model humans in the robotic cell and their interactions with objects. This would enable not only replanning under scene changes, but also probabilistic and contingent planning strategies to improve robustness, safety, and adaptability in human-robot industrial environments.

3. FRAMEWORK ARCHITECTURE AND EVALUATION

The agentic workflow shown in Fig. 1 is implemented using a LangChain-based architecture [53]. Operator interaction with the framework is supported by the graphical user interface shown in Fig. 4, which allows the user to provide natural-language task requests, optional contextual information, and execution feedback. The graph combines programmatic nodes, which perform deterministic operations such as parsing, compilation, validation, and planner invocation, with LLM-based nodes. In the figure, blocks denoted as

LLM Engine correspond to prompt-based LLM calls without external tools, whereas blocks denoted as *LLM Agent* correspond to prompt-based LLM calls augmented with tool interfaces, such as ROS 2 introspection tools. The code-generation module is implemented as an external service accessed through a web-server interface and is invoked when a missing PlanSys2-compatible action executor can be synthesized from the available ROS 2 interfaces. The experimental evaluation will compare different LLM backends in terms of problem-generation validity, repair success rate, planning success rate, plan quality, and execution-grounding success. Preliminary tests consider Llama 3.1-8B [54] for the pipeline up to symbolic plan generation. The planned evaluation will compare open-weight and closed-source LLM backends, including Llama 3.1, Qwen3, Mistral Large, Gemma 4, and GPT- and Gemini-family models, such as GPT-4.1, GPT-5.1, and Gemini 2.5 Pro [55–60]. The framework will first be evaluated in a simplified single-robot pick-and-place scenario, and then extended to multi-robot industrial-inspired tasks in order to assess the benefits of the proposed neuro-symbolic architecture in settings requiring symbolic reasoning, executable action grounding, and runtime capability discovery.

For Research Goal 2, the evaluation will also investigate the generation of a skill registry compliant with a predefined ontology. The purpose is to make the intermediate skill representation independent of a specific planning formalism, such as PDDL, and to enable the generation of the planning model that is most appropriate for the target task. This line of work is being developed in collaboration with ontology experts from ISTC-CNR, and is being integrated into a joint European project proposal for the EIC Pathfinder Challenge DeepRAP.¹

For Research Goal 3, the evaluation will focus on the use of Scene Graphs as explicit relational scene representations for mechanical assemblies. Initial experiments will investigate whether existing SGG architectures, such as REACT++, can be fine-tuned or adapted for real-time inference in this domain, measuring both relation-prediction accuracy and inference time. If this representation is insufficient, VLM-based alternatives (e.g. MomaGraph) will be considered and fine-tuned on assembly-specific data. For Behavior Tree generation, the evaluation will compare general-purpose LLMs conditioned on Scene Graph inputs, fine-tuned LLMs using structured Scene Graph representations, and existing BT-generation approaches such as BTGenBot-2 and multimodal Behavior Tree generation [52]. These results will also be compared against the proposed neuro-symbolic planning pipeline previously described.

4. EXPECTED SCIENTIFIC CONTRIBUTIONS

The project is expected to contribute to the following research directions:

- a neuro-symbolic architecture for natural-language robotic task programming;
- a method for generating PDDL problems from human requests while preserving planner-based verification;
- a computational-graph-based approach for grounding symbolic actions into executable robotic skills;
- a skill registry built from multimodal sources of information about the industrial process, serving as a knowledge base for the LLM and linking process knowledge to ROS 2 and PlanSys2 execution components, robot capabilities, and process constraints;
- methods for LLM-assisted PDDL domain generation from skills, documentation, and scene context;
- scene graph representations for mechanical assemblies and disassembly reasoning;
- a comparison between Behavior Tree generation and PDDL-based planning for robotic disassembly tasks.

5. ALIGNMENT WITH THE RESEARCH PROGRAM

The proposal is aligned with research activities on AI-based no-code robot programming, generative and deliberative AI integration, structured knowledge representations for planning, and deployment in ROS 2-based robotic systems. It also contributes to task planning, PlanSys2-based execution, skill and tool orchestration, and scene-graph-based knowledge extraction from the operational environment.

The project is particularly relevant to robotic systems that require a combination of natural-language interaction, symbolic reasoning, executable skill grounding, and safe integration with industrial robotic cells. Its long-term goal is to support robotic systems that can interpret human requests, reason over available capabilities and scene context, generate verifiable plans, and execute them through reliable robotic software frameworks.

6. CONCLUSION

This proposal outlines a research path toward neuro-symbolic robotic planning for no-code task programming. The central idea is to exploit large language models where they are most useful, namely natural-language interaction, knowledge extraction, and structured generation, while relying on symbolic planners for deterministic and explainable plan synthesis. The main contribution is the grounding layer between symbolic actions and executable robotic skills, based on computational-graph analysis, skill registries, and tool-based recovery of missing execution information.

The project further extends this framework by investigating automatic PDDL domain generation and scene graph reconstruction for mechanical disassembly. Together, these components aim to support robotic systems that are easier to program, more transparent to inspect, and more robust when deployed in realistic industrial environments.

REFERENCES

- [1] J. Liang *et al.*, *Code as policies: Language model programs for embodied control*, 2022. arXiv: 2209.07753 [cs.RO].
- [2] I. Singh *et al.*, “Progprompt: Generating situated robot task plans using large language models”, *Autonomous Robots*, vol. 47, pp. 999–1012, 2023.
- [3] M. Ahn *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances”, in *Proceedings of the 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205, 2023.
- [4] K. Lin, C. Agia, T. Migimatsu, M. Pavone, and J. Bohg, “Text2motion: From natural language instructions to feasible plans”, *Autonomous Robots*, 2023.
- [5] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, *et al.*, *Palm-e: An embodied multimodal language model*, 2023. arXiv: 2303.03378 [cs.LG].
- [6] B. Zitkovich *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control”, in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 2165–2183.
- [7] M. J. Kim *et al.*, “OpenVLA: An open-source vision-language-action model”, in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270, PMLR, 2025, pp. 2679–2713.
- [8] K. Black *et al.*, “ π_0 : A vision-language-action flow model for general robot control”, in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, Jun. 2025. DOI: 10.15607/RSS.2025.XXI.010.
- [9] Q. Shen *et al.*, *World action models: A survey*, 2026. arXiv: 2606.20781 [cs.RO].

¹https://eic.ec.europa.eu/eic-funding-opportunities/eic-pathfinder/eic-pathfinder-challenges-2026_en

- [10] A. Ham and M.-J. Park, “Human–robot task allocation and scheduling: Boeing 777 case study”, *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1256–1263, 2021.
- [11] Y. Liu, Z. Jiang, and C. Ke, “A task allocation method in human-robot collaboration (hrc) for the disassembly of automotive traction batteries”, in *2022 International Conference on Cyber-Physical Social Intelligence (ICCSI)*, 2022, pp. 332–337. DOI: [10.1109/ICCSI55536.2022.9970619](https://doi.org/10.1109/ICCSI55536.2022.9970619).
- [12] A. Umbrico *et al.*, “Design of advanced human–robot collaborative cells for personalized human–robot collaborations”, *Applied Sciences*, vol. 12, no. 14, p. 6839, 2022.
- [13] Y. Y. Liao and K. Ryu, “Task allocation in human-robot collaboration (hrc) based on task characteristics and agent capability for mold assembly”, *Procedia manufacturing*, vol. 51, pp. 179–186, 2020.
- [14] C. Tonola *et al.*, “Reactive and safety-aware path replanning for collaborative applications”, *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 21 456–21 468, 2025. DOI: [10.1109/TASE.2025.3607037](https://doi.org/10.1109/TASE.2025.3607037).
- [15] S. Sandrini, M. Faroni, and N. Pedrocchi, “Learning and planning for optimal synergistic human–robot coordination in manufacturing contexts”, *Robotics and Computer-Integrated Manufacturing*, vol. 95, p. 103 006, 2025, ISSN: 0736-5845. DOI: <https://doi.org/10.1016/j.rcim.2025.103006>.
- [16] K. Valmeekam, M. Marquez, A. Olmo, S. Sreedharan, and S. Kambhampati, *Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change*, 2022. arXiv: [2206.10498](https://arxiv.org/abs/2206.10498) [cs.AI].
- [17] K. Valmeekam, M. Marquez, S. Sreedharan, and S. Kambhampati, “On the planning abilities of large language models: A critical investigation”, in *Advances in Neural Information Processing Systems*, 2023.
- [18] M. Aghzal, E. Plaku, and Z. Yao, *Can large language models be good path planners? a benchmark and investigation on spatial-temporal reasoning*, 2023. arXiv: [2310.03249](https://arxiv.org/abs/2310.03249) [cs.RO].
- [19] K. Valmeekam, K. Stechly, and S. Kambhampati, *Llms still can't plan; can llms? a preliminary evaluation of openai's o1 on planbench*, 2024. arXiv: [2409.13373](https://arxiv.org/abs/2409.13373) [cs.AI].
- [20] S. Kambhampati *et al.*, “Llms can't plan, but can help planning in llm-modulo frameworks”, in *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [21] M. Chiari, L. Putelli, N. Rossetti, I. Serina, and A. Gerevini, “On planning through llms”, in *Proceedings of the Thirty-Fifth International Conference on Automated Planning and Scheduling*, 2025.
- [22] D. McDermott *et al.*, “Pddl – the planning domain definition language”, Yale Center for Computational Vision and Control, Tech. Rep. CVC TR-98-003/DCS TR-1165, 1998.
- [23] M. Fox and D. Long, “Pddl2.1: An extension to pddl for expressing temporal planning domains”, *Journal of Artificial Intelligence Research*, vol. 20, pp. 61–124, 2003.
- [24] D. E. Smith, J. Frank, and W. Cushing, “The anml language”, in *Proceedings of the ICAPS Workshop on Knowledge Engineering for Planning and Scheduling*, 2008.
- [25] S. Sanner, “Relational dynamic influence diagram language (rddl): Language description”, Australian National University, Tech. Rep., 2010.
- [26] J. Benton, A. Coles, and A. Coles, “Temporal planning with preferences and time-dependent continuous costs”, in *Proceedings of the Twenty-Second International Conference on Automated Planning and Scheduling*, 2012.
- [27] S. Panjkovic and A. Micheli, “Abstract action scheduling for optimal temporal planning via omt”, in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, 2024, pp. 20 222–20 229.
- [28] M. Cialdea Mayer, A. Orlandini, and A. Umbrico, “Planning and execution with flexible timelines: A formal account”, *Acta Informatica*, vol. 53, pp. 649–680, 2016.
- [29] A. Cimatti, M. Do, A. Micheli, M. Roveri, and D. E. Smith, “Strong temporal planning with uncontrollable durations”, *Artificial Intelligence*, vol. 256, pp. 1–34, 2018.
- [30] Mausam and D. S. Weld, “Planning with durative actions in stochastic domains”, *Journal of Artificial Intelligence Research*, vol. 31, pp. 33–82, 2008.
- [31] M. Tantakoun, X. Zhu, and C. Muise, “Llms as planning formalizers: A survey for leveraging large language models to construct automated planning models”, in *Findings of the Association for Computational Linguistics: ACL*, 2025.
- [32] B. Liu *et al.*, *Llm+p: Empowering large language models with optimal planning proficiency*, 2023. arXiv: [2304.11477](https://arxiv.org/abs/2304.11477) [cs.AI].
- [33] L. Guan, K. Valmeekam, S. Sreedharan, and S. Kambhampati, *Leveraging pre-trained large language models to construct and utilize world models for model-based task planning*, 2023. arXiv: [2305.14909](https://arxiv.org/abs/2305.14909) [cs.AI].
- [34] J. Oswald, S. Kambhampati, *et al.*, *Large language models as planning domain generators*, 2024. arXiv: [2405.06650](https://arxiv.org/abs/2405.06650) [cs.AI].
- [35] E. Gestrin, G. Kuhlmann, and J. Seipp, *Nl2plan: Robust llm-driven planning from minimal text descriptions*, 2024. arXiv: [2405.04215](https://arxiv.org/abs/2405.04215) [cs.AI].
- [36] S. Li *et al.*, “Safe planner: Empowering safety awareness in large pre-trained models for robot task planning”, in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, 2025, pp. 14 619–14 627.
- [37] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Sünderhauf, “Sayplan: Grounding large language models using 3d scene graphs for scalable robot task planning”, in *Proceedings of the Conference on Robot Learning*, 2023.
- [38] J. Herzog, J. Liu, and Y. Wang, *Domain-conditioned scene graphs for state-grounded task planning*, 2025. arXiv: [2504.06661](https://arxiv.org/abs/2504.06661) [cs.RO].
- [39] Z. Yang, C. Garrett, D. Fox, T. Lozano-Pérez, and L. P. Kaelbling, “Guiding long-horizon task and motion planning with vision language models”, in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2025.
- [40] R. Howey, D. Long, and M. Fox, “Val: Automatic plan validation, continuous effects and mixed initiative planning using pddl”, in *Proceedings of the IEEE International Conference on Tools with Artificial Intelligence*, 2004.
- [41] A. Micheli *et al.*, “Unified planning: Modeling, manipulating and solving ai planning problems in python”, *SoftwareX*, vol. 29, p. 102 012, 2025, ISSN: 2352-7110. DOI: <https://doi.org/10.1016/j.softx.2024.102012>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2352711024003820>.
- [42] C. Garrett and F. Ramos, “ScheduleStream: Temporal planning with samplers for GPU-accelerated multi-arm task and motion planning & scheduling”, in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [43] M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI: An Introduction*. Boca Raton, FL, USA: CRC Press, 2018. DOI: [10.1201/9780429489105](https://doi.org/10.1201/9780429489105).

- [44] O. Biggar, M. Zamani, and I. Shames, “An expressiveness hierarchy of behavior trees and related architectures”, *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5397–5404, 2021. DOI: [10.1109/LRA.2021.3074337](https://doi.org/10.1109/LRA.2021.3074337).
- [45] G. Zhu *et al.*, “Scene graph generation: A comprehensive survey”, *arXiv preprint arXiv:2201.00443*, 2022. DOI: [10.48550/arXiv.2201.00443](https://doi.org/10.48550/arXiv.2201.00443).
- [46] M. Neau, P. E. Santos, A.-G. Bossier, A. Sugimoto, and C. Buche, “React: Real-time efficiency and accuracy compromise for trade-offs in scene graph generation”, in *36th British Machine Vision Conference 2025, BMVC 2025, Sheffield, UK, November 24-27, 2025*, BMVA, 2025. [Online]. Available: https://bmva-archive.org.uk/bmvc/2025/assets/papers/Paper_239/paper.pdf.
- [47] M. Neau and Z. Falomir, “REACT++: Efficient cross-attention for real-time scene graph generation”, *arXiv preprint arXiv:2603.06386*, 2026. DOI: [10.48550/arXiv.2603.06386](https://doi.org/10.48550/arXiv.2603.06386).
- [48] Y. Ju *et al.*, “MomaGraph: State-aware unified scene graphs with vision-language model for embodied task planning”, *arXiv preprint arXiv:2512.16909*, 2025. DOI: [10.48550/arXiv.2512.16909](https://doi.org/10.48550/arXiv.2512.16909).
- [49] J. Wald, H. Dhano, N. Navab, and F. Tombari, “Learning 3d semantic scene graphs from 3d indoor reconstructions”, in *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [50] R. A. Izzo, G. Bardaro, and M. Matteucci, “BTGenBot: Behavior tree generation for robotic tasks with lightweight LLMs”, *arXiv preprint arXiv:2403.12761*, 2024.
- [51] R. A. Izzo, G. Bardaro, and M. Matteucci, “BTGenBot-2: Efficient behavior tree generation with small language models”, *arXiv preprint arXiv:2602.01870*, 2026.
- [52] C. Battistini, R. A. Izzo, G. Bardaro, and M. Matteucci, “Multi-modal behavior tree generation: A small vision-language model for robot task planning”, *arXiv preprint arXiv:2603.06084*, 2026.
- [53] H. Chase, *Langchain*, 2022. [Online]. Available: <https://github.com/langchain-ai/langchain>.
- [54] Meta AI, *Introducing Llama 3.1: Our most capable models to date*, <https://ai.meta.com/blog/meta-llama-3-1/>, Accessed: 2026-07-09, 2024.
- [55] A. Yang *et al.*, “Qwen3 technical report”, *arXiv preprint arXiv:2505.09388*, 2025.
- [56] Mistral AI, *Large enough: Mistral Large 2*, <https://mistral.ai/news/mistral-large-2407/>, Accessed: 2026-07-09, 2024.
- [57] Google DeepMind, *Gemma 4: Open models for advanced reasoning and agentic workflows*, <https://deepmind.google/models/gemma/gemma-4/>, Accessed: 2026-07-10, 2026.
- [58] OpenAI, *GPT-4.1 model*, <https://developers.openai.com/api/docs/models/gpt-4.1>, Accessed: 2026-07-09, 2025.
- [59] OpenAI, *GPT-5.1 model*, <https://developers.openai.com/api/docs/models/gpt-5.1>, Accessed: 2026-07-09, 2025.
- [60] Google, *Gemini 2.5 Pro*, <https://ai.google.dev/gemini-api/docs/models/gemini-2.5-pro>, Accessed: 2026-07-10, 2025.